

A Controlled Study of Pure Monte Carlo Tree Search for Symbolic Regression

After conversations with GPT

August 4, 2026

1 A controlled study of pure Monte Carlo tree search for symbolic regression

1.1 Problem state for symbolic regression

Question 1.1. What is our problem set up?

Suppose (x_i, y_i) are a set of points generated from an expression

$$h(x_i) = y_i$$

for a fixed expression h (unknown), h is a quadratic equation.

The object being optimized: is the discrete set of symbolic structures, over

$$\mathcal{T} = \mathcal{P}(\{1, x, x^2\})$$

.

Symbolic regression search of the structures τ : We have an expression :

$$\tau_\beta(x) = \beta_0 + \beta_1(x) + \beta_2x^2$$

we have the variables $1, x, x^2$ (discrete set) , $\beta = (\beta_1, \beta_2, \beta_3)$ are continuous.

We will fit

$$\hat{\beta}_\tau = \arg \min_{\beta} \frac{1}{n} \sum_{i=1}^n (y_i - \tau_\beta(x_i))^2$$

Symbolic search find all $\tau \in \mathcal{T}$ (structures)

$$\tau^* = \arg \min_{\tau \in \mathcal{T}} R(\tau, \hat{\beta}_\tau)$$

where R is R^2 (why?)

Question 1.2. What simplifications should we make to see the impact of MCTS?

Some simplification we should make in the first experimental design:

- no variable length trees.
- no invalid expressions.

- no noise.
- no complex penalty.
- no additional operator.
- No neural policy or value network.

So we pretty much turn the problem into a *subset selection*

The purpose of this study is to characterize the finite-budget limitations of pure Monte Carlo tree search (MCTS) in grammar-based symbolic regression. We deliberately exclude neural policy and value networks. The neural-guidance question is scientifically separate: before asking whether a network improves search, one must determine what information the underlying pure-search procedure estimates and when that information is aligned with symbolic optimization.

1.2 The derivation MDP

Question 1.3. What exactly is the Markov decision process that the search runs on?

Everything below (5) presumes a state space of partial derivations, a legal-action set, a deterministic transition and a reward that arrives only at termination. We now fix those objects, because the implementation we audit realizes them concretely and the realization is not quite the idealization.

States. A state is a *sentential form*: a finite string over the grammar’s alphabet, some of whose symbols are nonterminals. Write

$$\mathcal{S} = \{s \in \Sigma^{\leq L-1}\}, \quad s_0 = S, \quad (1)$$

where Σ is the symbol alphabet, S the start symbol and L a token budget. The implementation stores s as a fixed-length buffer of L integers padded on the right, together with the count of occupied slots; the pad symbol is what makes the observation space a fixed-shape array rather than a variable-length string. A state is terminal exactly when it contains no nonterminal.

Actions. An action names *which* nonterminal to rewrite and *how*:

$$\mathcal{A}(s) = \{(i, p) : s_i \in \mathcal{N}, p \in \text{Prod}(s_i), |s| + |\text{rhs}(p)| - 1 < L\}, \quad (2)$$

with $\mathcal{N}$ the nonterminals and $\text{Prod}(A)$ the productions of A . Pairs are flattened to a single index $a = iP + p$, where P is the total number of productions, so the action space is $\{0, \dots, LP - 1\}$ with (2) supplied as a Boolean mask. Two features of (2) are easy to miss. First, the length test is strict, so a finished expression has at most $L - 1$ tokens: the budget L is one token larger than the longest reachable expression. Second, the position i is chosen by the agent. The derivation order is therefore *not* fixed, and $|\mathcal{A}(s)|$ grows with the number of open nonterminals rather than being a constant $|\text{Prod}|$; the six states of Fig. 1(a) have $|\mathcal{A}| = 4, 8, 12, 8, 4, 0$.

Transition. Rewriting is splicing, and it is deterministic:

$$T(s, (i, p)) = s_{0:i} \cdot \text{rhs}(p) \cdot s_{i+1:|s|}. \quad (3)$$

Every step either lengthens the buffer or closes one nonterminal without lengthening it, so an episode terminates in at most $2L$ steps.

Reward. Intermediate rewards vanish. A terminal string π is read as a prefix-notation expression whose $\mathbb{C}_k$ tokens are free constants; those constants are fitted to the data by least squares and the reward is the resulting coefficient of determination,

$$R(\pi) = \text{clip}_{[-1,1]} \left(1 - \frac{\text{SS}_{\text{res}}(\pi)}{\text{SS}_{\text{tot}}} \right), \quad R(s) = 0 \text{ for } s \text{ nonterminal.} \quad (4)$$

The clip is not cosmetic: it is what stops an unparseable or singular expression from returning $-\infty$ and poisoning every value that backs up through it. It also means the reward range is $[-1, 1]$, not the $[0, 1]$ that (17) arranges for the controlled family.

Completion policy and budget. The rollout policy is uniform on (2), $q(a \mid s) = 1/|\mathcal{A}(s)|$, and since (3) is deterministic it induces $q(\cdot \mid s) \in \Delta(\mathcal{E}(s))$ exactly as in (6). Note that this is uniform on *pairs*: a form with two open nonterminals splits its mass across positions, so q is not the per-slot uniform law of (24). One simulation performs one terminal evaluation, so the budget B of (45) is the simulation count of a single persistent root search — measured, B simulations at one completion per expanded leaf cost exactly $B + 1$ terminal evaluations, the extra one being the root expansion.

Table 1 collects the seven components and their realization.

Component	Formal object	Realization
State	sentential form $s \in \mathcal{S}$, (1); $s_0 = S$	length- L integer buffer, right-padded, plus an occupied-slot count. Terminal iff no nonterminal remains
Action	(i, p) : position and production, (2); flattened to $a = iP + p$	$\{0, \dots, LP - 1\}$ with a Boolean legality mask. The agent chooses <i>which</i> nonterminal to expand, so the derivation order is free
Transition	$T(s, (i, p))$, splice, (3); deterministic	in-place buffer splice; the strict length test in (2) caps finished expressions at $L - 1$ tokens
Reward	0 on nonterminals; $R(\pi)$ at a leaf, (4)	prefix $\rightarrow$ infix $\rightarrow$ symbolic parse $\rightarrow$ least-squares fit of the free constants $\rightarrow R^2$, clipped to $[-1, 1]$; any failure scores -1 . Memoized per expression string
Termination	$\mathcal{A}(s) = \emptyset$	no nonterminal in the occupied prefix; at most $2L$ steps
Completion policy	$q(a \mid s) = 1/ \mathcal{A}(s) $	uniform over the flattened mask, i.e. over <i>pairs</i> (i, p) and not over the productions of one designated slot
Budget	B terminal evaluations, (45)	B simulations of one persistent root search, one completion per expanded leaf: exactly $B + 1$ evaluations

Table 1: The seven components of the derivation MDP and how each is realized. The three entries that do not simply instantiate the formalism — a free derivation order, a reward on $[-1, 1]$ rather than $[0, 1]$, and a completion law uniform over position–production pairs — are the ones Remark 1.4 takes up.

Remark 1.4 (Where the formalism and the implementation diverge). Three of the seven rows of Table 1 are not instantiations of the formalism but departures from it, and each one has a consequence for the experiments.

1. *The derivation order is free.* The controlled grammar of (24) fills four labelled slots, so a completion is a tuple $\mathbf{j}$ and the uniform law factorizes. Here the agent also chooses i , so $|\mathcal{A}(s)|$ is state-dependent and q does not factorize. V^q and ρ_0 must then be computed by recursion over sentential forms, as in Fig. 1(b), rather than by the closed forms (32) and (34).

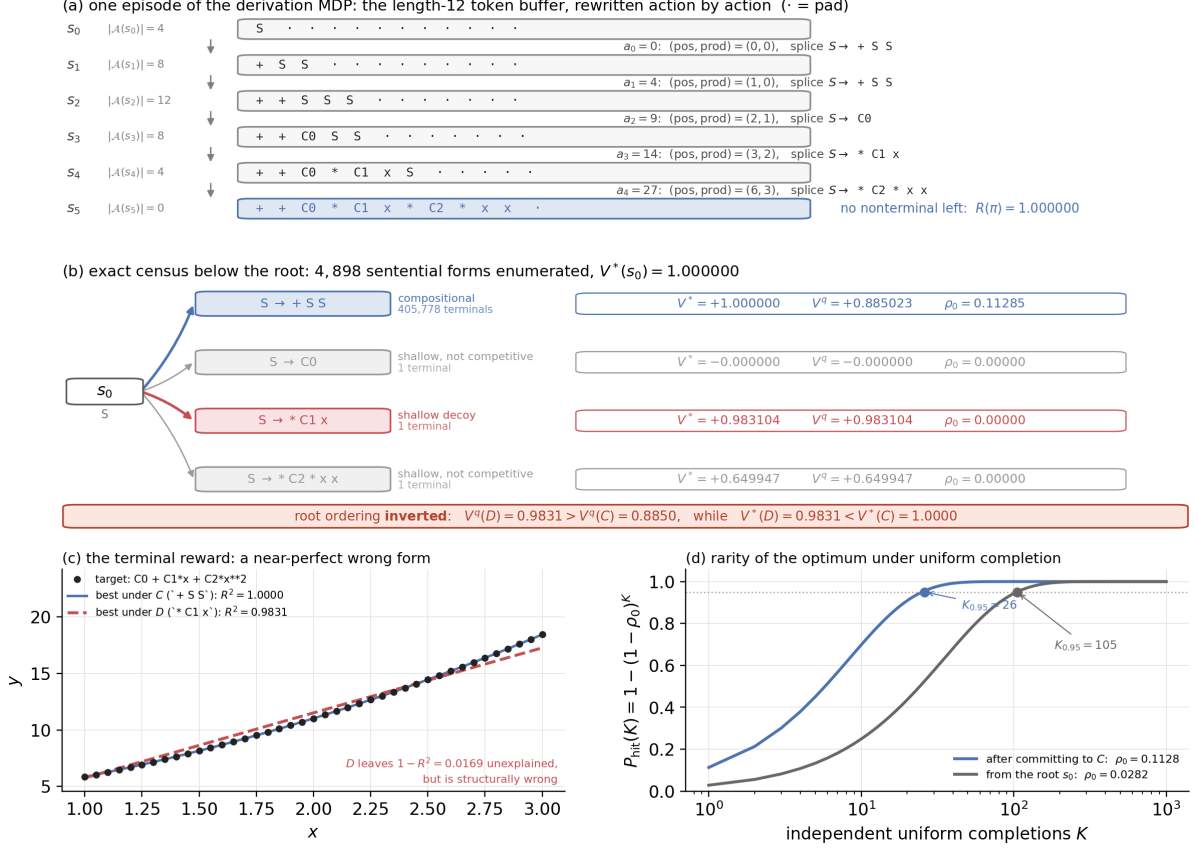


Figure 1: The derivation MDP on the additive-quadratic instance, target $y = c_0 + c_1x + c_2x^2$ on 41 points of $[1, 3]$, token budget $L = 12$. Colours follow Fig. 3: red is the shallow branch with the highest reachable reward, blue the compositional branch that contains the optimum, grey the remaining shallow branches. (a) One episode: the token buffer of (1) rewritten by the splice (3), with the flat action index, its decoding into (position, production), and $|\mathcal{A}(s_k)|$ at each step. Note that $|\mathcal{A}|$ rises to 12 and falls back: it counts open nonterminals, not productions. (b) The exact census. Enumerating all 4,898 mask-reachable sentential forms gives V^* , V^q and ρ_0 of (39) at every state without sampling. The three shallow branches each hold one terminal, so $V^q = V^*$ there; the compositional branch holds 405,778. (c) The reward (4) in data space: the best one-term expression reaches $R^2 = 0.9831$ on a structurally wrong form, which is the exact trap of Remark 2.2 arising unprompted. (d) The hit curve (35) at the measured ρ_0 . Generated by `figures/make_mdp_figures.py`.

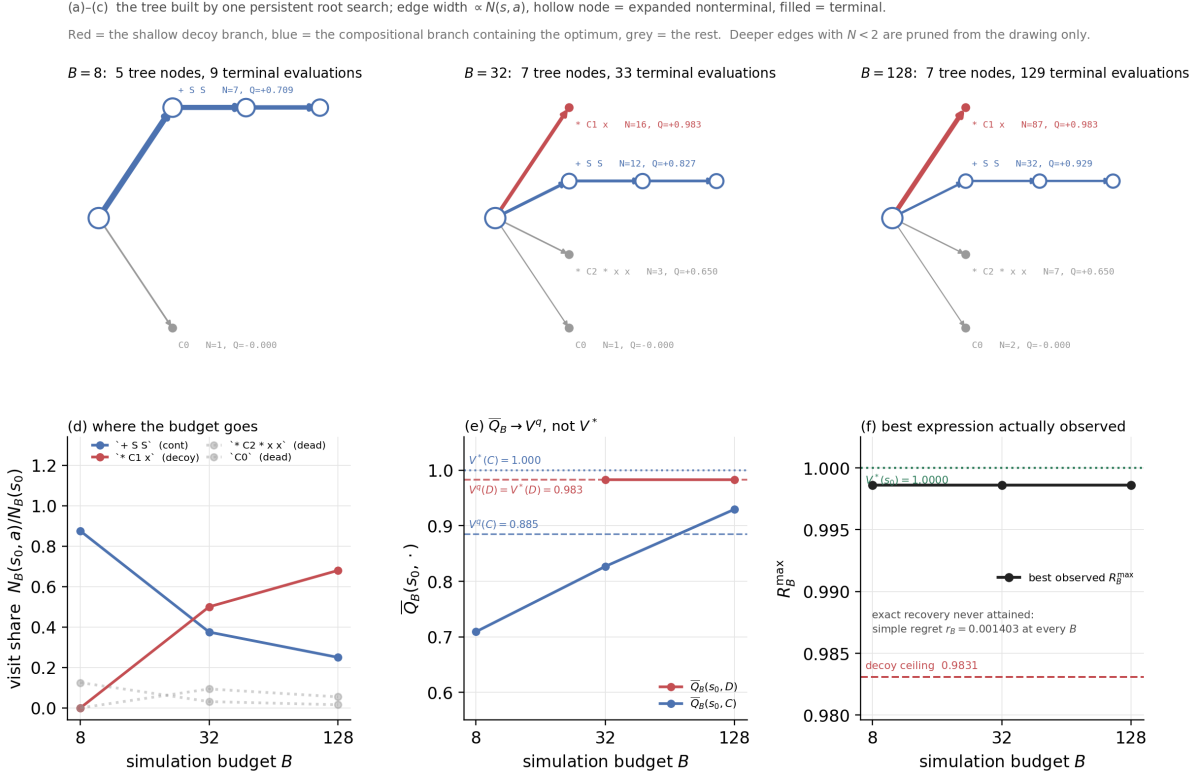


Figure 2: The same environment searched by pure MCTS — uniform prior, one random completion per expanded leaf, mean backup — from a single persistent root, at budgets $B \in \{8, 32, 128\}$. **(a)–(c)** The tree actually built. Edge width is the visit count $N(s, a)$; the labels are read off the search, not assigned. **(d)** The visit share of each root action. It moves the wrong way: the branch containing the optimum takes 0.875 of the budget at $B = 8$ and only 0.25 at $B = 128$, while the shallow decoy climbs to 0.68. **(e)** Why. The exploitation statistic $\bar{Q}_B$ converges to V^q , not V^* , and here $V^q(D) = 0.983 > V^q(C) = 0.885$ while $V^*(D) = 0.983 < V^*(C) = 1$: the value-semantic failure of Table 2, measured rather than constructed. **(f)** The consequence. The best expression observed is 0.998597 at every budget — the search never recovers the exact three-term form, so simple regret is small while exact recovery fails, the separation (47) is designed to expose. Generated by `figures/make_mdp_figures.py`.

2. *The reward lives on $[-1, 1]$.* The clip in (4) is a guard against fit failure, and it costs the identity (17): a reward of -1 is not a variance share. The exploration constant of (42), which is the Hoeffding deviation for rewards on $[0, 1]$, is then no longer calibrated. This is one reason the controlled family is worth building separately rather than reusing this instance.
3. *Constants are fitted nonlinearly.* Under (24) the model is linear in its coefficients and (27) is a projection. Here a general Levenberg–Marquardt solve is run from a fixed initialization under an evaluation cap, so the reward is a property of the *oracle* as much as of the structure: the same expression can score differently under a different cap. The 10^{-12} cross-check demanded by Experiment 1 is therefore a check against a linear-algebra evaluator, not against this one.

None of the three affects the definitions (5) and (6), which need only a deterministic transition and a terminal reward. What they affect is which closed forms apply, and that is exactly the gap the controlled family of (12) is built to close.

1.3 Motivation and scope

Question 1.5. What problems do we want to identify?

The central distinction is between the quality of a typical random completion and the quality of the best reachable completion.

Let s be a partial, derivation, let $\mathcal{A}(s)$ be set of its legal actions, let $T(s, a)$ be the deterministic transition function.

$$T : \mathcal{S} \times \mathcal{A} \rightarrow \mathcal{S}$$

A complete sequence of actions, produce a terminal expression π , with reward $R(\pi)$.

Question 1.6. Can we design problems where it is easy to analytically evaluate $R(\pi)$.

$$\mathcal{E}(s) := \{\pi : \pi \text{ can be completed from } s\}$$

denote the set of terminal expressions reachable from s .

Question 1.7. What is the best expression hidden below the state?

The best-achievable value is

$$V^*(s) = \max_{\pi \in \mathcal{E}(s)} R(\pi), \tag{5}$$

where $R(\pi)$ is the terminal symbolic-regression reward.

Question 1.8. In MCTS we use random roll out: what is the typical uniformly random completion under this state?

Consider the fixed random-completion policy that, at each nonterminal state s , selects uniformly among the legal actions $\mathcal{A}(s)$,

$$q(a \mid s) = \frac{1}{|\mathcal{A}(s)|}, \quad a \in \mathcal{A}(s).$$

Because transitions are deterministic, following a uniform policy from s to termination induces a distribution $q(\pi \mid s)$ over the completions $\pi \in \mathcal{E}(s)$, i.e.

$$q(-|s) \in \text{Dist}(\mathcal{E}(s)).$$

The mean-rollout value is

$$V^q(s) = \mathbb{E}_{\pi \sim q(\cdot|s)} [R(\pi)]. \quad (6)$$

Remark 1.9. Why do we consider uniform distribution: MCTS has 4 stage:

1. select (start at root) Choose action according to UCT formula

$$a^* \in \arg \max_{a \in \mathcal{A}(s)} \left[Q(s, a) + c \sqrt{\frac{\log N(s)}{N(s, a)}} \right]$$

2. Let s be the state which reach at the "end" of the selection phase. We have an expansion policy : which is a uniform roll out from s to obtain the value $V^q(s)$.
3. Value backup: ...

Remark 1.10. The optimization objective depends on V^* , whereas conventional UCT uses empirical mean returns as its exploitation statistic [8]. Asymptotic consistency does not by itself imply effective finite-budget search, and adverse finite-depth behavior is known for UCT [3]. This distinction is particularly relevant to symbolic regression, where the goal is to discover at least one exceptional expression rather than a policy with high average return. Best-focused Monte Carlo methods have a history in single-player search and expression discovery [1, 2], while risk-seeking symbolic regression explicitly optimizes upper-tail rather than expected performance [10]. Recent MCTS-based symbolic-regression systems also emphasize learned proposal mechanisms or extreme-value allocation rather than pure mean-based UCT [6, 7].

Thoughts. We believe there are two potential problems that can make pure MCTS bad:

1. *value-semantic failure*: V^q ranks competing branches in the opposite order from V^* ; formally, for two sibling states s_1 and s_2 , a value-semantic failure exists if:

$$V^q(s_1) > V^q(s_2) \quad \text{while} \quad V^*(s_1) < V^*(s_2)$$

In this scenario, MCTS confidently exploits the wrong branch because a shallow, suboptimal "decoy" yields a high expected reward, while the branch containing the true global optimum is penalized by a low expected reward due to the abundance of poor completions.

2. *rare-event failure*: near-optimal completions have very small probability under the rollout policy. For a tolerance $\varepsilon \geq 0$, this is the probability that a random rollout yields an expression within ε of the true optimum:

$$\rho_\varepsilon(s) = \Pr_{\pi \sim q(\cdot|s)} [R(\pi) \geq V^*(s) - \varepsilon]$$

where $q(\cdot \mid s)$ is the uniform rollout.

If this probability is low: then this means that the near optimal completions are rare events. This is what lead to the max operator in the UCT formula.

Example 1.11. The probability that the random rollout gets an optimal reward is:

$$\rho_0(s) = \Pr_{\pi \sim q(\cdot|s)}[R(\pi) = V^*(s)]$$

Therefore: in any experiments, we should study

$$V^q(s), V^*(s), \rho_\varepsilon(s)$$

as our three primary metrics.

Table 2 summarizes the two mechanisms.

[milton: We can emperically look at public symbolic regression problems and verify whether these two failures modes are there.]

[milton: Question for AI: are these metrics sufficient? Is it practical? can we give explicit metrics for practical problems?]

Failure mechanism	Formal condition	Intuitive cause	Implication for search
Value-semantic failure	$V^q(s_1) > V^q(s_2)$ while $V^*(s_1) < V^*(s_2)$ for sibling states s_1, s_2	A suboptimal branch has the higher mean rollout reward: a shallow decoy looks better	Search confidently exploits the wrong branch and starves the branch containing the true global optimum.
Rare-event failure	$\rho_\varepsilon(s) \ll B^{-1}$	Near-optimal completions carry a negligible fraction of the rollout probability mass.	Search explores the correct branch but exhausts the budget B before sampling a near-optimal leaf.

Table 2: The two finite-budget failure mechanisms of mean-based MCTS.

[milton: The strategy of the project :

1. Design a family of problems (y_λ) (the λ is what dictates the difficulty of symbolic regression problem) Such that failure modes occur.
2. See if MCTS succeed or not when these failure modes are present.
3. Understand if neural net acutally help.

|

Experiment 1 measures these two quantities exactly. Experiment 2 tests whether they predict the finite-budget behavior of pure MCTS.

Our goal is then to answer:

Question 1.12. After identifying the difficulty in pure MCTS, how does neural net address these?

1.4 Empirical function space

Suppose we are given to functions $f, g : \mathbb{R} \rightarrow \mathbb{R}$. We want to compute the empeirical mean and variance on the set of points.

$$x_1, \dots, x_n \in [-1, 1], \quad n = 41,$$

be equally spaced sample points,

We want to compute empirical mean and or variance of ¹

$$\{f(x_i)\}_{i=1}^n$$

We define the empirical inner product

Definition 1.13 (Empirical inner product). Define the inner product $\langle -, - \rangle_n : \mathbb{R}^n \times \mathbb{R}^n \rightarrow \mathbb{R}$ given by

$$\langle (x_i), (y_i) \rangle_n = \frac{1}{n} \sum_{i=1}^n x_i y_i, \quad (7)$$

Two functions agreeing on the grid are identified, so (7) is an inner product on $\mathbb{R}^n$ and every statement below is Euclidean geometry in $\mathbb{R}^n$. Let $\mathbf{1}$ denote the constant function 1 and write

$$\|f\|_n^2 = \langle f, f \rangle_n = \frac{1}{n} \sum_{i=1}^n f(x_i)^2, \quad \bar{f} = \langle f, \mathbf{1} \rangle_n = \frac{1}{n} \sum_{i=1}^n f(x_i), \quad \text{Var}_n(f) = \|f - \bar{f} \mathbf{1}\|_n^2. \quad (8)$$

Three readings of these symbols are used repeatedly.

1. $\|f\|_n^2$ is the *mean* square of f over the grid, not the sum of squares; the factor $1/n$ is what makes $\mathbf{1}$ a unit vector, $\|\mathbf{1}\|_n = 1$. The two conventions differ by a constant, $\|f\|_2^2 = n\|f\|_n^2$.
2. $\bar{f}$ is at once the sample mean of f and the coordinate of f along $\mathbf{1}$; hence $\bar{f} \mathbf{1}$ is the orthogonal projection of f onto $\text{span}\{\mathbf{1}\}$, and f is centered exactly when $f \perp \mathbf{1}$.
3. Splitting $\mathbb{R}^n = \text{span}\{\mathbf{1}\} \oplus \mathbf{1}^\perp$ gives the Pythagorean identity

$$\|f\|_n^2 = \bar{f}^2 + \text{Var}_n(f), \quad (9)$$

so squared norm and empirical variance coincide precisely on centered functions.

Starting from the monomials $x, x^2, \dots, x^{12}$, remove their empirical means and apply Gram–Schmidt orthogonalization,

$$\psi_1, \dots, \psi_{12}$$

satisfying

$$\langle \psi_j, \mathbf{1} \rangle_n = 0, \quad \langle \psi_j, \psi_\ell \rangle_n = \delta_{j\ell}. \quad (10)$$

The first two basis functions are the normalized linear and centered-quadratic components,

$$\psi_1(x) = \frac{x}{\sqrt{\mu_2}}, \quad \psi_2(x) = \frac{x^2 - \mu_2}{\sqrt{\nu}}, \quad (11)$$

where

$$\mu_2 = \langle x, x \rangle_n, \quad \nu = \langle x^2 - \mu_2, x^2 - \mu_2 \rangle_n.$$

Thus the construction extends the orthogonal linear–quadratic decomposition used in the motivating quadratic example.

1.5 Controlled target family

Question 1.14. What family of target functions should we consider?

- What discrete set $\mathcal{T}$ (of expressions, $\psi_1, \dots, \psi_M$) should we consider? Let suppose

¹we think of these as the y_i s from the target functions .

suppose our expressions $\mathcal{T}$ has only 4 generators, $\psi_1, \psi_2, \psi_3, \psi_4$.

$$\mathcal{T} = \mathcal{P}(\{\psi_1, \psi_2, \psi_3, \psi_4\})$$

- How should we choose the $\psi_1, \dots, \psi_4$. We want to choose monomials x, x^2, x^3, x^4 , however, for computational purposes, as we will require easy to compute variance, we will do some simplifications on the monomials with respect to the given dataset $\{x_i\}_{i=1}^n$ as an input to our symbolic regression problem. The process is [Section 1.4](#).

For $\lambda \in (0, 1)$, define the four-component target

$$y_\lambda(x) = \sqrt{\lambda} \psi_1(x) + \sqrt{\frac{1-\lambda}{3}} (\psi_2(x) + \psi_3(x) + \psi_4(x)). \quad (12)$$

where these ψ_i s are the orthonormal basis functions constructed above. The strategy now:

1. explicitly compute $V^q(s)$ and $V^*(s)$ $\rho_\varepsilon(s)$, for each state s when we run the symbolic game with the target y_λ .

Thoughts. You can assume these ψ_i s are x^i .

Proposition 1.15 (Zero mean and unit target variance). For every $\lambda \in (0, 1)$,

$$\bar{y}_\lambda = \langle y_\lambda, 1 \rangle_n = 0, \quad \|y_\lambda\|_n^2 = \langle y_\lambda, y_\lambda \rangle_n = 1. \quad (13)$$

Proof. Both identities use only the three properties of the basis (10): each ψ_j is centered, $\langle \psi_j, 1 \rangle_n = 0$; has unit length, $\langle \psi_j, \psi_j \rangle_n = 1$; and is orthogonal to the others, $\langle \psi_j, \psi_\ell \rangle_n = 0$ for $j \neq \ell$. For the mean, linearity gives

$$\langle y_\lambda, 1 \rangle_n = \sqrt{\lambda} \langle \psi_1, 1 \rangle_n + \sqrt{\frac{1-\lambda}{3}} \sum_{m=2}^4 \langle \psi_m, 1 \rangle_n = 0.$$

Since the mean is zero, $\|y_\lambda\|_n^2$ is the empirical variance of y_λ . Expanding $\langle y_\lambda, y_\lambda \rangle_n$ produces sixteen terms; orthogonality annihilates every cross-term, and unit length reduces each surviving square to its coefficient:

$$\|y_\lambda\|_n^2 = \lambda + 3 \cdot \frac{1-\lambda}{3} = 1.$$

The identity is the Pythagorean theorem in the geometry (7). □

The parameter λ is therefore exactly the fraction of target variance carried by ψ_1 . We use

$$\lambda_{\text{weak}} = \frac{1}{4}, \quad \lambda_{\text{strong}} = 0.997. \quad (14)$$

When $\lambda = 1/4$, all four target components have equal variance share. When $\lambda = 0.997$, the first component explains 99.7% of the target variance and each remaining component explains 0.1%. The latter is a controlled analogue of a pseudo-linear target: a shallow one-term expression is almost perfectly predictive but is not structurally exact. [Figures 3](#) and [4](#) display the two scenarios end to end, from the variance ledger to the root values.

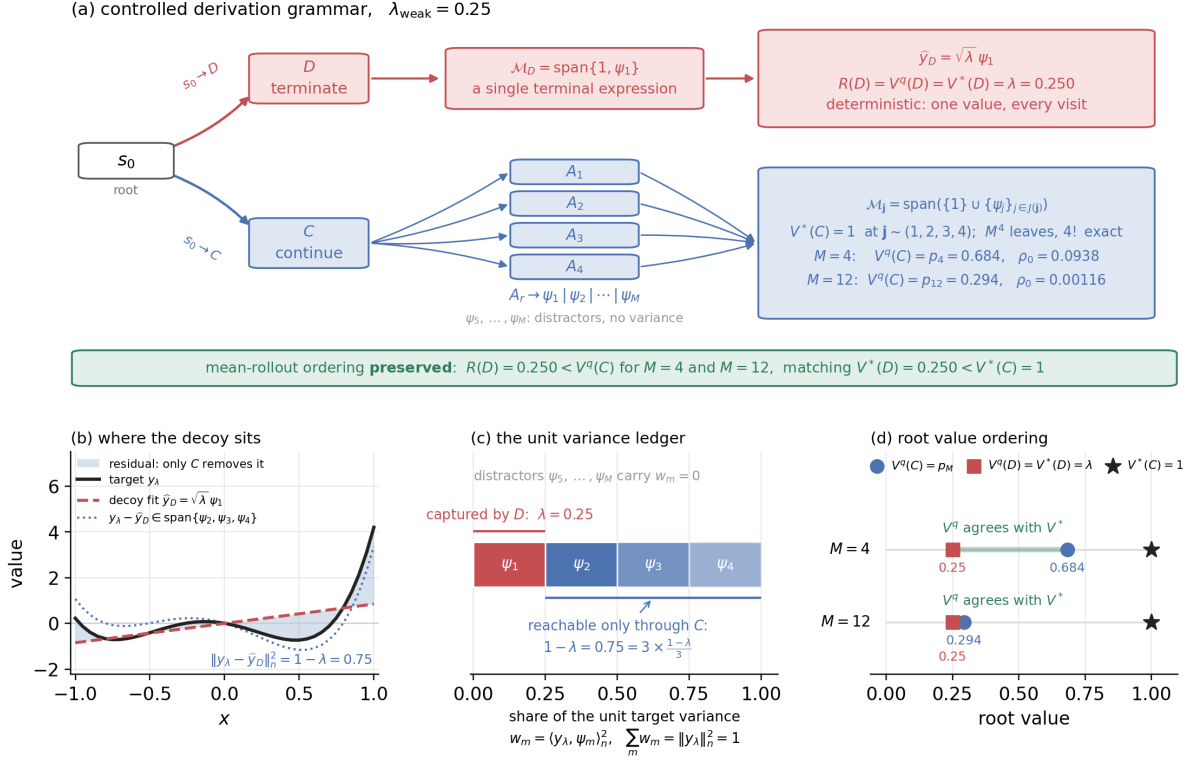


Figure 3: The weak-decoy scenario, $\lambda_{\text{weak}} = 1/4$. One colour per role throughout: red is everything the decoy action D reaches, blue is everything reachable only through the continue action C , grey the distractor atoms $\psi_5, \dots, \psi_M$. (a) The controlled derivation grammar, with the exact values (30), (31), (32) and (34) printed on the leaves; D terminates immediately into $\mathcal{M}_D$, while C opens the four atom slots $A_1, \dots, A_4$ of (24). (b) The same picture in function space: the decoy fit $\hat{y}_D = \sqrt{\lambda} \psi_1$ leaves the residual $\sqrt{(1-\lambda)/3}(\psi_2 + \psi_3 + \psi_4)$, of squared length $1 - \lambda = 0.75$. (c) The variance ledger (19): the unit target variance of Proposition 1.15 splits into four equal shares, of which D can capture only the first. (d) Root values. The mean-rollout evaluator ranks C above D at both M , so it agrees with the V^* ordering and no value-semantic failure occurs; only rare-event failure remains, and it is governed by ρ_0 . Generated by `figures/make_scenario_figures.py`.

1.6 Why the target is normalized

Proposition 1.15 is a normalization. Two elementary facts make this precise: the normalization costs no generality (Lemma 1.16), and it converts the reward into an additive accounting of target variance (Lemma 1.17). Throughout, $\mathcal{M} \subseteq \mathbb{R}^n$ is a linear model class containing $\mathbf{1}$, $\Pi_{\mathcal{M}}$ is the orthogonal projection onto $\mathcal{M}$ in the geometry (7) — equivalently, the least-squares fit (27) — and

$$R(y; \mathcal{M}) = 1 - \frac{\|y - \Pi_{\mathcal{M}}y\|_2^2}{\|y - \bar{y}\mathbf{1}\|_2^2} \quad (15)$$

is the unpenalized R^2 reward of (28). Th

Lemma 1.16 (The reward is affine-invariant). For all $\alpha \neq 0$ and $\beta \in \mathbb{R}$,

$$R(\alpha y + \beta \mathbf{1}; \mathcal{M}) = R(y; \mathcal{M}). \quad (16)$$

Proof. Since $\Pi_{\mathcal{M}}$ is linear and $\mathbf{1} \in \mathcal{M}$, we have

$$\Pi_{\mathcal{M}}(\alpha y + \beta \mathbf{1}) = \alpha \Pi_{\mathcal{M}}y + \beta \mathbf{1}$$

where we note that $\Pi_{\mathcal{M}}(\mathbf{1}) = \mathbf{1}$.

so the residual scales by α ,

$$(\alpha y + \beta \mathbf{1}) - \Pi_{\mathcal{M}}(\alpha y + \beta \mathbf{1}) = \alpha (y - \Pi_{\mathcal{M}}y).$$

The centered target scales by α as well, because $\overline{\alpha y + \beta} = \alpha \bar{y} + \beta$. Numerator and denominator of (15) therefore both acquire the factor α^2 , which cancels. $\square$

Consequently the reward landscape — and with it V^* , V^q and ρ_ε at every state of the search tree — is constant on each affine family $\{\alpha y + \beta \mathbf{1} : \alpha \neq 0, \beta \in \mathbb{R}\}$. Every nonconstant family contains exactly two centered unit-norm representatives, $\pm(y - \bar{y}\mathbf{1})/\sqrt{\text{Var}_n(y)}$, and (12) selects one of them. Normalizing is thus a choice of coordinates on an equivalence class, and it buys the following identity.

Lemma 1.17 (The reward is captured variance). Suppose $\bar{y} = 0$ and $\|y\|_n = 1$, and let $\{\mathbf{1}, \phi_1, \dots, \phi_k\}$ be an orthonormal basis of $\mathcal{M}$ in the geometry (7). Then

$$R(y; \mathcal{M}) = 1 - \|y - \Pi_{\mathcal{M}}y\|_n^2 = \|\Pi_{\mathcal{M}}y\|_n^2 = \sum_{i=1}^k \langle y, \phi_i \rangle_n^2 \in [0, 1]. \quad (17)$$

Proof. The two norms differ by a constant, $\|\cdot\|_2^2 = n\|\cdot\|_n^2$, so the factor n cancels in (15); since $\bar{y} = 0$ the denominator is $n\|y\|_n^2 = n$, which gives the first equality. The residual is orthogonal to $\mathcal{M}$, hence to $\Pi_{\mathcal{M}}y$, so Pythagoras gives $1 = \|y\|_n^2 = \|\Pi_{\mathcal{M}}y\|_n^2 + \|y - \Pi_{\mathcal{M}}y\|_n^2$, which is the second. Expanding the projection in the orthonormal basis and using $\langle y, \mathbf{1} \rangle_n = \bar{y} = 0$ leaves $\Pi_{\mathcal{M}}y = \sum_i \langle y, \phi_i \rangle_n \phi_i$, and Parseval gives the third. Finally $0 \leq \|\Pi_{\mathcal{M}}y\|_n^2 \leq \|y\|_n^2 = 1$. $\square$

Write

$$w_m = \langle y, \psi_m \rangle_n^2, \quad m = 1, \dots, M, \quad (18)$$

for the share of target variance carried by the m th atom. In this notation Proposition 1.15 states exactly that

$$w = (\lambda, \frac{1-\lambda}{3}, \frac{1-\lambda}{3}, \frac{1-\lambda}{3}, 0, \dots, 0) \quad \text{is a probability vector on } \{1, \dots, M\}, \quad (19)$$

and Lemma 1.17, applied to the model class (26), reads

$$R(\mathbf{j}) = w(J(\mathbf{j})) = \sum_{m \in J(\mathbf{j})} w_m. \quad (20)$$

The reward of a terminal expression is the w -measure of the set of atoms it selects. This is (29), and it is what the normalization is for. Four consequences are used throughout.

1. *The design parameter is the decoy reward.* For the unnormalized target $\sqrt{\lambda} \psi_1 + c(\psi_2 + \psi_3 + \psi_4)$ the decoy captures the fraction $\lambda/(\lambda + 3c^2)$. Only the constraint $\lambda + 3c^2 = 1$ makes the coefficient we set equal to the number we compare, so that $R(D) = \lambda$ (Proposition 2.1) and the inversion condition reads $\lambda > p_M$ in (33) with no conversion factor. Had $\|y_\lambda\|_n$ drifted with λ , turning the decoy knob would also have rescaled every reward, confounding the value-semantic effect with a scale effect.
2. *One absolute reward scale across the four cells.* By (17), $R \in [0, 1]$ with $R = 1$ if and only if $J(\mathbf{j}) \supseteq \{1, 2, 3, 4\}$, so exact recovery is a reward value rather than a tolerance, and simple regret $r_B = 1 - R_B^{\max}$ is the unexplained share of target variance. In particular $1 - \lambda_{\text{strong}} = 0.003$ is literally 0.3% of the target variance. Cells differing in λ and M are therefore compared on the same axis, and the UCT bonus in (42), whose constant is the Hoeffding deviation for rewards supported in $[0, 1]$, is admissible in all of them without recalibration.
3. *Exact statistics become counting.* Because (20) is additive in the selected atoms, taking expectations reduces V^q to four marginal inclusion probabilities; this is precisely the proof of Proposition 2.3, and it requires neither sampling nor fitting. The same identity is what the 10^{-12} agreement between the analytic and least-squares evaluators tests, the denominator of (28) being the constant n rather than a fitted quantity.
4. *Centering is what makes “variance share” true.* Since $\bar{y}_\lambda = 0$ and every model class considered contains $\mathbf{1}$, the intercept vanishes identically, least squares reduces to coordinatewise projection onto the selected ψ_j , and $\|y_\lambda\|_n^2 = \text{Var}_n(y_\lambda)$ by (9). Without centering, the R^2 denominator would no longer be $\|y_\lambda\|_n^2$ and λ would not be the fraction of variance explained by ψ_1 .

Remark 1.18. Lemma 1.16 also delimits the claim. Normalization fixes bookkeeping, not difficulty: it changes no reward, no ordering and no completion probability. The controlled family (12) is therefore not a special case but a canonical representative, and the secondary validation on centered, variance-normalized quadratic targets tests the same reward landscape as the raw ones.

2 Controlled derivation grammar

The root state s_0 has two legal actions.

Decoy action. The first action, denoted D , terminates immediately with the shallow model class

$$\mathcal{M}_D = \text{span}\{1, \psi_1\}. \quad (21)$$

The branch contains a single terminal expression, fitted by least squares and scored by the same unpenalized R^2 criterion as every terminal expression, cf. (28) below. Every visit therefore returns one deterministic reward, and that reward is exactly the decoy parameter.

Proposition 2.1 (Decoy reward). For every $\lambda \in (0, 1)$, the least-squares fit of y_λ in $\mathcal{M}_D$ is

$$\hat{y}_D = \sqrt{\lambda} \psi_1, \quad (22)$$

and its reward is

$$R(D) = \lambda. \quad (23)$$

Proof. Because the numerator and denominator of (28) carry the same factor n , the ratio is unchanged if both are computed in the geometry (7); we do so throughout. The constant function satisfies $\langle 1, 1 \rangle_n = 1$, so by (10) the set $\{1, \psi_1\}$ is an orthonormal basis of $\mathcal{M}_D$, and the least-squares fit is the orthogonal projection

$$\hat{y}_D = \langle y_\lambda, 1 \rangle_n 1 + \langle y_\lambda, \psi_1 \rangle_n \psi_1.$$

The intercept vanishes, $\langle y_\lambda, 1 \rangle_n = 0$, by Proposition 1.15. For the slope, expand y_λ by (12); orthogonality annihilates the three cross-terms and unit length leaves

$$\langle y_\lambda, \psi_1 \rangle_n = \sqrt{\lambda} \langle \psi_1, \psi_1 \rangle_n = \sqrt{\lambda},$$

which proves (22). The residual is then explicit,

$$y_\lambda - \hat{y}_D = \sqrt{\frac{1-\lambda}{3}} (\psi_2 + \psi_3 + \psi_4),$$

and the Pythagorean theorem in the geometry (7) gives $\|y_\lambda - \hat{y}_D\|_n^2 = 3 \cdot \frac{1-\lambda}{3} = 1 - \lambda$. Since $\bar{y}_\lambda = 0$ and $\|y_\lambda\|_n^2 = 1$ (Proposition 1.15), the reward (28) evaluates to

$$R(D) = 1 - \frac{\|y_\lambda - \hat{y}_D\|_n^2}{\|y_\lambda\|_n^2} = 1 - (1 - \lambda) = \lambda. \quad \square$$

Remark 2.2 (The decoy is an exact trap). Proposition 2.1 is classical: on orthonormal regressors, least squares projects coordinatewise and R^2 is the captured share of target variance [5, §3.2.3]. Its role here is structural. At $\lambda_{\text{strong}} = 0.997$, a structurally incomplete one-term model earns a deterministic reward of 0.997 in exact arithmetic, while the sibling branch containing the exact expression returns the far smaller mean (32) under random completion. Objectives whose locally averaged signal points away from the global optimum are called *deceptive* in evolutionary computation [4]; the analogous hazard for sampling-based tree search is the shallow *search trap* [11], on which UCT’s transitory regret can grow doubly exponentially in the depth [3]. The decoy transplants this structure into single-player symbolic regression, where a near-perfect R^2 on a wrong functional form is precisely the failure mode that motivates reporting predictive-accuracy and symbolic solutions separately in benchmark evaluation [9].

Continue action. The second action, denoted C , opens the compositional branch. It contains the structurally exact four-component expression, whose reward is 1; a uniform random completion, however, recovers that expression only rarely, and the mean completion reward is p_M , cf. (32).

The action creates four atom slots $A_1, \dots, A_4$. Each slot has the productions

$$A_r \longrightarrow \psi_1 \mid \psi_2 \mid \dots \mid \psi_M, \quad r = 1, \dots, 4, \quad (24)$$

where

$$M \in \{4, 12\}. \quad (25)$$

A terminal continuation is an ordered tuple

$$\mathbf{j} = (j_1, j_2, j_3, j_4) \in \{1, \dots, M\}^4.$$

Let

$$J(\mathbf{j}) = \{j_1, j_2, j_3, j_4\}$$

be the set of distinct selected atoms. The corresponding fitted model space is

$$\mathcal{M}_{\mathbf{j}} = \text{span}(\{1\} \cup \{\psi_j : j \in J(\mathbf{j})\}). \quad (26)$$

Repeated atoms are removed before fitting, or equivalently handled by the Moore–Penrose pseudoinverse.

For a design matrix $X_{\mathbf{j}}$, the fitted coefficients are

$$\hat{a}_{\mathbf{j}} = (X_{\mathbf{j}}^T X_{\mathbf{j}})^{\dagger} X_{\mathbf{j}}^T y_{\lambda}. \quad (27)$$

The terminal reward is unpenalized R^2 ,

$$R(\mathbf{j}) = 1 - \frac{\|y_{\lambda} - X_{\mathbf{j}} \hat{a}_{\mathbf{j}}\|_2^2}{\|y_{\lambda} - \bar{y}_{\lambda} \mathbf{1}\|_2^2}. \quad (28)$$

No reward clipping or nonlinear optimizer is used.

Repeating the projection argument in the proof of Proposition 2.1, now for the orthonormal basis of $\mathcal{M}_{\mathbf{j}}$, shows that the reward has the exact form

$$R(\mathbf{j}) = \lambda \mathbf{1}\{1 \in J(\mathbf{j})\} + \frac{1 - \lambda}{3} \sum_{m=2}^4 \mathbf{1}\{m \in J(\mathbf{j})\}. \quad (29)$$

Consequently, all rewards lie in $[0, 1]$, distractor atoms $\psi_5, \dots, \psi_M$ capture no target variance, and the exact reward is independent of numerical fitting pathologies.

2.1 Exact completion statistics

Let D and C denote the root children corresponding to the decoy and continue actions. Since the decoy child admits the single completion of Proposition 2.1, it satisfies

$$V^*(D) = V^q(D) = \lambda. \quad (30)$$

Proposition 2.3 (Continue-branch values). The continue branch selects four atoms uniformly and independently from the vocabulary $\{\psi_1, \dots, \psi_M\}$. Its best-achievable reward is

$$V^*(C) = 1, \quad (31)$$

and its mean-rollout reward is

$$V^q(C) = p_M = 1 - \left(1 - \frac{1}{M}\right)^4. \quad (32)$$

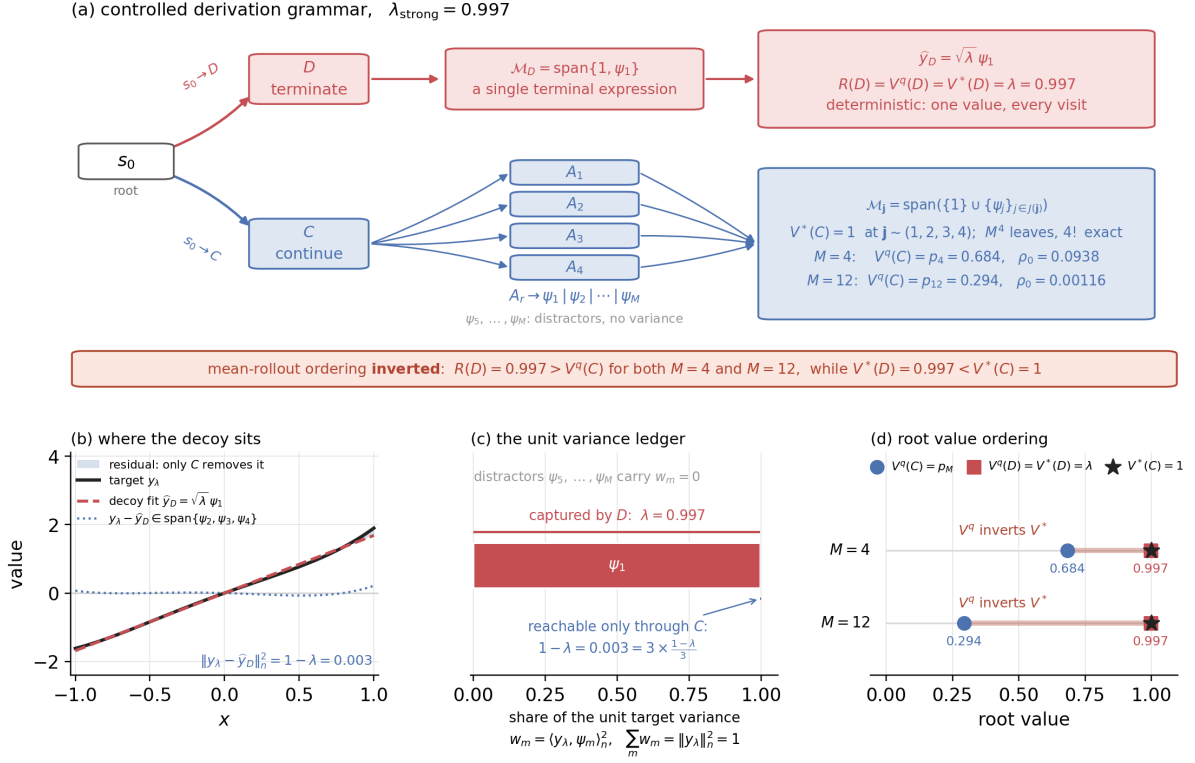


Figure 4: The strong-decoy scenario, $\lambda_{\text{strong}} = 0.997$. Colours and panels are as in Fig. 3; only λ has changed, and by Lemma 1.16 nothing else about the environment has been rescaled. (b) The decoy fit is now visually indistinguishable from the target and the residual is flat at the plotting scale, yet it is not zero: $\|y_\lambda - \hat{y}_D\|_n^2 = 1 - \lambda = 0.003$. This is the pseudo-linear regime, where near-perfect prediction coexists with structural error. (c) The decoy now captures 99.7% of the unit variance, and the three shares reachable only through C are 0.1% each. (d) The mean-rollout evaluator reverses the correct ordering at both M , since $R(D) = 0.997 > p_M$ while $V^*(D) = 0.997 < V^*(C) = 1$; this is the value-semantic failure of Table 2 and the exact trap of Remark 2.2.

Proof. The reward (29) is maximal when $J(\mathbf{j}) \supseteq \{1, 2, 3, 4\}$, where every indicator equals one and

$$R(\mathbf{j}) = \lambda + \frac{1-\lambda}{3} \cdot 3 = 1;$$

the tuple $\mathbf{j} = (1, 2, 3, 4)$ achieves this, proving (31). For the mean, take expectations in (29) under the uniform completion policy q :

$$V^q(C) = \lambda \Pr_q(1 \in J(\mathbf{j})) + \frac{1-\lambda}{3} \sum_{m=2}^4 \Pr_q(m \in J(\mathbf{j})).$$

A fixed atom m is missed by one slot with probability $1 - \frac{1}{M}$, hence by all four independent slots with probability $(1 - \frac{1}{M})^4$, so $\Pr_q(m \in J(\mathbf{j})) = p_M$ for every m . Substituting gives $V^q(C) = \lambda p_M + (1-\lambda)p_M = p_M$. $\square$

Comparing (30) with (32), the mean-rollout evaluator reverses the correct root ordering exactly when

$$\lambda > p_M. \quad (33)$$

Exact recovery requires the four selected atoms to form a permutation of $(1, 2, 3, 4)$. Among the M^4 equally likely ordered tuples, exactly $4!$ are permutations of the target, hence

$$\rho_0(C) = \Pr_q(R = 1 \mid C) = \frac{4!}{M^4}. \quad (34)$$

Question 2.4. What is the minimum number of rollouts (K) we must perform to guarantee, with 95% confidence, that we will see the exact solution at least once?

Proposition 2.5 (Hit probability and sample complexity). Draw K independent uniform completions of C . The probability that at least one is exact is

$$P_{\text{hit}}(K) = 1 - (1 - \rho_0(C))^K, \quad (35)$$

and, for $\delta \in (0, 1)$, the guarantee $P_{\text{hit}}(K) \geq 1 - \delta$ holds if and only if

$$K \geq K_{1-\delta} = \left\lceil \frac{\log \delta}{\log(1 - \rho_0(C))} \right\rceil. \quad (36)$$

Proof. A single completion misses the exact expression with probability $1 - \rho_0(C)$. By independence, all K completions miss with probability $(1 - \rho_0(C))^K$, and the complement of that event is (35). The guarantee $P_{\text{hit}}(K) \geq 1 - \delta$ is therefore equivalent to

$$(1 - \rho_0(C))^K \leq \delta.$$

Taking logarithms gives $K \log(1 - \rho_0(C)) \leq \log \delta$, and dividing by $\log(1 - \rho_0(C))$, which is negative, reverses the inequality:

$$K \geq \frac{\log \delta}{\log(1 - \rho_0(C))}.$$

The smallest integer satisfying this bound is the ceiling in (36). $\square$

Table 3 gives the four environments.

3 Experiment 1: exact value-semantic audit

Cell	λ	M	$V^q(C)$	$\rho_0(C)$	Inversion	$K_{0.95}$
Weak decoy, low distraction	0.25	4	0.683594	0.09375	No	31
Strong decoy, low distraction	0.997	4	0.683594	0.09375	Yes	31
Weak decoy, high distraction	0.25	12	0.293933	0.001157	No	2587
Strong decoy, high distraction	0.997	12	0.293933	0.001157	Yes	2587

Table 3: Controlled 2×2 design. The decoy parameter λ controls whether the random-completion mean has the correct root ordering, while M controls the mass of exact completions. The column $K_{0.95}$ is conditional on repeatedly sampling the continue branch.

Question 3.1. What is the relation between these two values and running MCTS in practice?

1. Does mean random-completion return preserve the ordering induced by best-achievable reward i.e V^q and V^* rank the states correctly ?
2. and how rare is the exact expression within each partial derivation?

$$\rho_\varepsilon$$

The steps of the full experiment:

1. Define family y_λ .
2. Plot the values of primary outputs.

Hypothesis 1. The rollout mean ranks the continue branch above the decoy for $\lambda = 0.25$ and below the decoy for $\lambda = 0.997$:

$$V^q(C) > \lambda_{\text{weak}}, \quad V^q(C) < \lambda_{\text{strong}}. \quad (37)$$

Independently, the completion budget required for exact recovery scales as

$$K_{1-\delta} = \Theta\left(\frac{\log(1/\delta)}{\rho_0}\right) \quad \text{as } \rho_0 \rightarrow 0. \quad (38)$$

Procedure. No stochastic tree search is used in this experiment. For each of the four cells, enumerate all M^4 terminal continuations and all partial prefixes of depths 0, 1, 2, 3, 4. For each partial state s , compute exactly

$$V^*(s), \quad V^q(s), \quad \rho_0(s). \quad (39)$$

For a nonterminal state s , define the action selected by the mean-rollout oracle as

$$a_q(s) = \arg \max_a V^q(T(s, a)), \quad (40)$$

and define its oracle sibling regret by

$$g_q(s) = V^*(s) - V^*(T(s, a_q(s))). \quad (41)$$

The largest environment contains only

$$1 + 12 + 12^2 + 12^3 + 12^4 = 22,621$$

continue-prefix states, so exhaustive calculation is tractable.

The implementation must compute each terminal reward in two ways:

1. using the analytical expression (29);
2. using the actual least-squares evaluator (27)–(28).

The two values must agree to numerical tolerance 10^{-12} .

Primary outputs. Report the exact root quantities

$$V^*(D), \quad V^q(D), \quad V^*(C), \quad V^q(C), \quad \rho_0(C), \quad K_{0.95}.$$

Plot:

1. the terminal completion-reward distribution in each cell;
2. $V^q(s)$ against $V^*(s)$, stratified by state depth;
3. sibling regret $g_q(s)$ against depth;
4. the exact hit curve $1 - (1 - \rho_0)^K$.

Because Experiment 1 is an exact census, it requires no random seeds, confidence intervals, or significance tests. Failure to reproduce the analytical identities indicates an implementation error and should prevent proceeding to Experiment 2.

4 Experiment 2: finite-budget pure-MCTS limits

Question. At matched terminal-evaluation budgets, do the value-order mismatch and exact-completion rarity identified in Experiment 1 predict the behavior of actual pure search?

4.0.1 Search conditions

We compare three conditions.

Uniform random search. Sample complete derivations independently from the uniform grammar policy and return the best terminal expression observed.

Mean-UCT. At node s , select

$$a_t = \arg \max_a \left[\bar{Q}_t(s, a) + \sqrt{\frac{2 \log N_t(s)}{N_t(s, a)}} \right], \quad (42)$$

where $\bar{Q}_t(s, a)$ is the empirical mean of terminal returns from simulations passing through (s, a) . Unvisited actions receive infinite score. Each simulation selects through the current tree, expands one unvisited edge, performs one uniform random completion, and backs up the terminal reward by an arithmetic mean.

Max-UCT diagnostic. Define

$$Q_t^{\max}(s, a) = \max \{R(\pi_b) : \pi_b \text{ was sampled through } (s, a)\}. \quad (43)$$

Select using

$$a_t = \arg \max_a \left[Q_t^{\max}(s, a) + \sqrt{\frac{2 \log N_t(s)}{N_t(s, a)}} \right]. \quad (44)$$

The tree structure, expansion policy, rollout policy, and exploration term are otherwise identical to Mean-UCT. This condition is a causal diagnostic of the mean-versus-maximum distinction, not a claim that naive maximum backup is universally optimal. A calibrated extreme-bandit allocation rule is a natural subsequent comparison [6].

4.0.2 Evaluation protocol

Each search run uses one persistent tree rooted at s_0 . The algorithm does not commit to a grammar action before the budget is exhausted. No network, learned prior, Dirichlet noise, reward clipping, complexity penalty, or nonlinear optimizer is used. One simulation contains exactly one terminal reward evaluation.

Use the terminal-evaluation budgets

$$\mathcal{B} = \{16, 32, 64, 128, 256, 512, 1024, 2048, 4096, 8192\}. \quad (45)$$

For each of the four benchmark cells and each search condition, use 100 independent seeds. Run each seed once to $B_{\max} = 8192$ and record all intermediate checkpoints. The same base-seed list is used across algorithms.

4.0.3 Outcomes

Let $\pi_1, \dots, \pi_B$ be the terminal expressions observed by budget B . Define the best-observed reward

$$R_B^{\max} = \max_{1 \leq b \leq B} R(\pi_b), \quad (46)$$

the simple regret

$$r_B = 1 - R_B^{\max}, \quad (47)$$

and the exact-recovery indicator

$$S_B = \mathbf{1} \{R_B^{\max} \geq 1 - 10^{-12}\}. \quad (48)$$

The time to exact recovery is

$$T_{\text{exact}} = \inf\{B : S_B = 1\}. \quad (49)$$

Exact recovery is the primary outcome. Simple regret is secondary because the incorrect decoy in the strong-decoy cells already has simple regret

$$1 - \lambda_{\text{strong}} = 0.003.$$

Thus predictive reward alone cannot distinguish near-perfect approximation from structural recovery. This distinction is consistent with symbolic regression benchmarks that report both predictive and exact-recovery performance [9].

For mechanism diagnosis, record the continue-visit fraction

$$F_B = \frac{N_B(s_0, C)}{N_B(s_0)}, \quad (50)$$

together with the final root recommendation, number of distinct terminal sequences, number of distinct canonical atom sets, and the best reward found through each root action. The final visit-count recommendation is diagnostic only; the primary search output is the best terminal expression encountered.

4.0.4 Hypotheses

Hypothesis 2a: value-semantic effect. Mean-UCT will allocate less search to the continue branch under $\lambda = 0.997$ than under $\lambda = 0.25$, because its exploitation statistic converges toward $V^q(C)$, which lies below the strong-decoy reward. Max-UCT will be less sensitive to this inversion after observing high-reward continuations.

Hypothesis 2b: rarity effect. For every algorithm and fixed λ , exact recovery will be slower for $M = 12$ than for $M = 4$. At moderate budgets, recovery will be organized approximately by the effective rare-event budget

$$B\rho_0. \quad (51)$$

Hypothesis 2c: interaction. The largest recovery difference between Max-UCT and Mean-UCT will occur in the strong-decoy, high-distraction cell

$$(\lambda, M) = (0.997, 12),$$

where the continue branch has both a lower rollout mean than the decoy and a small exact-completion mass.

One pre-specified interaction contrast is

$$\Delta_{\text{semantic}}(B, M) = [P_B^{\text{Max}} - P_B^{\text{Mean}}]_{\lambda=0.997} - [P_B^{\text{Max}} - P_B^{\text{Mean}}]_{\lambda=0.25}, \quad (52)$$

where

$$P_B^A = \Pr_A(S_B = 1)$$

is the exact-recovery probability for algorithm A . The prediction is

$$\Delta_{\text{semantic}}(B, M) > 0,$$

with a larger effect expected at $M = 12$.

4.0.5 Statistical reporting and interpretation

For each budget, cell, and algorithm, report the exact-recovery proportion with a 95% Wilson interval, median simple regret with a seed-bootstrap interval, the empirical survival curve

$$\Pr(T_{\text{exact}} > B),$$

and the continue-visit fraction F_B .

The interpretation is mechanism-based:

- If Mean-UCT fails while Max-UCT succeeds, the mean-return statistic is causally implicated.
- If both methods deteriorate primarily when $M = 12$, completion rarity is the dominant limitation.
- If random search outperforms Mean-UCT in a strong-decoy cell, the tree allocation rule actively suppresses the optimal branch.
- If Mean-UCT succeeds despite the exact value-order inversion, UCT exploration is sufficient at the tested budgets and the mismatch alone is not a complete explanation.
- If Max-UCT performs worse in weak-decoy cells, maximum backup is overoptimistic and should not be treated as a universal replacement for mean backup.
- If simple regret is small while exact recovery remains poor, the experiment demonstrates a separation between predictive approximation and symbolic identification.

4.1 Secondary validation and deferred questions

After the two primary experiments, a nonconfirmatory validation may repeat the key comparison on centered and variance-normalized versions of the motivating quadratic targets, using an additive grammar and direct linear least squares. The original full grammar, nonlinear constant fitting, uniform-prior PUCT, learned policy and value heads, and SRBench evaluation should be introduced only after the controlled mechanisms are understood.

In particular, a subsequent neural experiment should ask whether a learned policy or value function corrects the pure-search failures identified here, or instead learns and amplifies the biased targets produced by mean-return search. Separating that learning question from the present pure-search study is necessary for a causal interpretation.

References

- [1] Tristan Cazenave. Nested monte-carlo search. In *Proceedings of the 21st International Joint Conference on Artificial Intelligence*, pages 456–461, 2009.
- [2] Tristan Cazenave. Monte-carlo expression discovery. *International Journal on Artificial Intelligence Tools*, 22(1):1250035, 2013.
- [3] Pierre-Arnaud Coquelin and Rémi Munos. Bandit algorithms for tree search. In *Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence*, pages 67–74. AUAI Press, 2007.
- [4] David E. Goldberg. Simple genetic algorithms and the minimal, deceptive problem. In Lawrence Davis, editor, *Genetic Algorithms and Simulated Annealing*, pages 74–88. Morgan Kaufmann, Los Altos, CA, 1987.
- [5] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, NY, second edition, 2009.
- [6] Zhengyao Huang, Daniel Zhengyu Huang, Tiannan Xiao, Dina Ma, Zhenyu Ming, Hao Shi, and Yuanhui Wen. Improving monte carlo tree search for symbolic regression. In *Advances in Neural Information Processing Systems*, volume 38, 2025.
- [7] Pierre-Alexandre Kamienny, Guillaume Lample, Sylvain Lamprier, and Marco Virgolin. Deep generative symbolic regression with Monte-Carlo-tree-search. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 15655–15668. PMLR, 2023.
- [8] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *Machine Learning: ECML 2006*, volume 4212 of *Lecture Notes in Computer Science*, pages 282–293. Springer, 2006.
- [9] William La Cava, Patryk Orzechowski, Bogdan Burlacu, Fabrício Olivetti de França, Marco Virgolin, Ying Jin, Michael Kommenda, and Jason H. Moore. Contemporary symbolic regression methods and their relative performance. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, volume 1, 2021.
- [10] Brenden K. Petersen, Mikel Landajuela, T. Nathan Mundhenk, Claudio P. Santiago, Soo K. Kim, and Joanne T. Kim. Deep symbolic regression: Recovering mathematical expressions from data via risk-seeking policy gradients. In *International Conference on Learning Representations*, 2021.

- [11] Raghuram Ramanujan, Ashish Sabharwal, and Bart Selman. On adversarial search spaces and sampling-based planning. In *Proceedings of the Twentieth International Conference on Automated Planning and Scheduling*, pages 242–245, 2010.